For any suggestions or feedback regarding these notes, please contact [Pragy Agarwal]

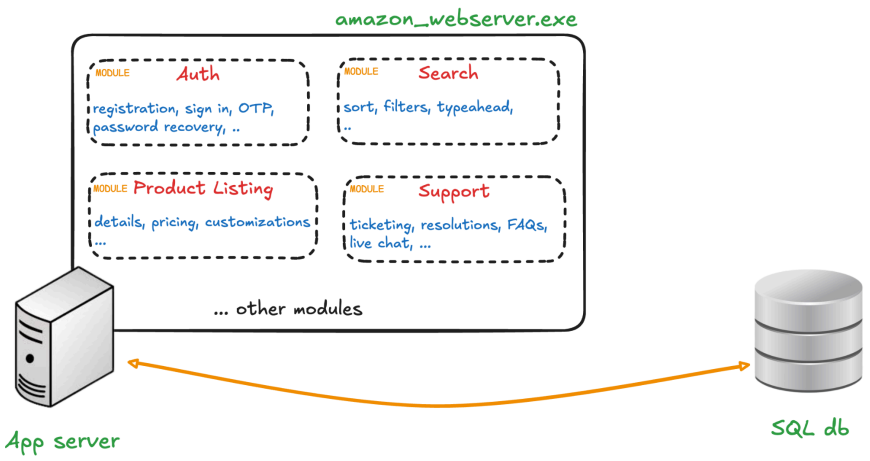
Communication

Once we split an application into multiple services, how these services talk to each other (and to clients) becomes a critical aspect of the architecture.

Monolith Communication

In monolithic systems, communication between components occurs through **simple function calls** within the same process.

- extremely performant communication
- 0 n/w overhead for communication



Microservice Communication

Microservices run in isolated processes or containers, communicating primarily over network protocols such as HTTP, RPC, or message brokers. This introduces multiple challenges:

- **Latency:** Network calls add delay (compared to function calls in monolith)
  - in practice, networks and frameworks are fast enough for most use cases (milliseconds of overhead)
    - especially true for cloud infrastructure within the same availability zone
    - most likely, if you're deployed in a given availability zone, all your servers are co-located (in a single data center)
    - and they're connected via a very high speed (10Gbps), low latency (<1ms) fiber network.
  - sometimes aggregation is needed (use an API Gateway to batch multiple calls into one).
    - this makes the latency worse
- **Reliability:** Network can fail, so we must implement retries (*so that we still get the job done)* and circuit breakers *(to isolate the failure)*

- **Versioning:** APIs evolve, and we must maintain backward compatibility.
    - you can slowly phase out old API versions
        - release a new version which breaks backwards compatibility
        - recommend everyone to upgrade to this new version, but, you will continue supporting the older versions
        - you mark the older version as “deprecated”
        - after a few more versions, you can actually remove this deprecated version, because most of the clients have (hopefully) now upgraded.
- **Consistency:** since the data is split across multiple services, we need to consider consistency

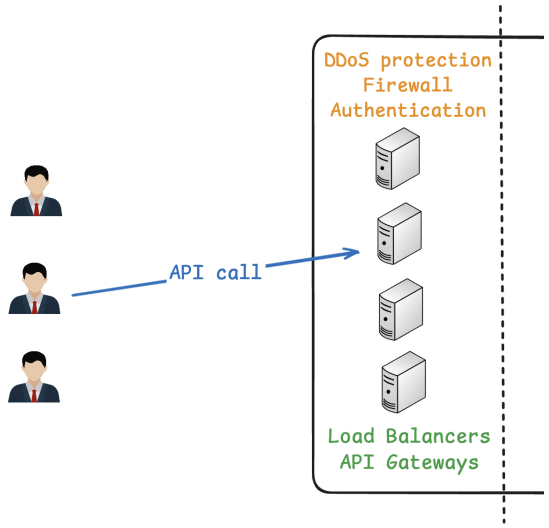
Clients ⇔ Microservices Communication

Clients communicate with the backend microservices via API calls.

1. API Gateway — most popular

These calls must pass through an API Gateway to be routed to the correct microservice.

The API gateway (or the untrusted n/w layer) can handle cross-cutting concerns like authentication, rate limiting, SSL termination, request aggregation, and more. The client just calls one endpoint (the gateway), which proxies or orchestrates calls to microservices behind the scenes.



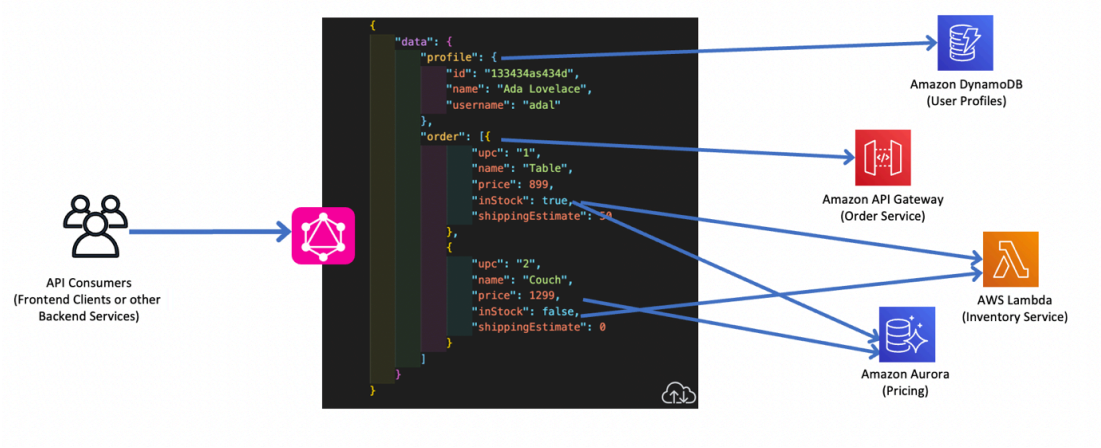
Popular open-source API gateways: **NginX**, netflix/**Zuul**, **Kong**, **Traefik**, **HAproxy**

2. GraphQL

solves the *(n+1) query problem*
allows the frontend to evolve independently of the backend

A query language that lets clients request exactly the data they need. The server (gateway or specialized GraphQL service) fetches from microservices and returns a structured JSON.

This can reduce over-fetching or under-fetching that can happen with REST. However, it requires a GraphQL schema and resolver logic.

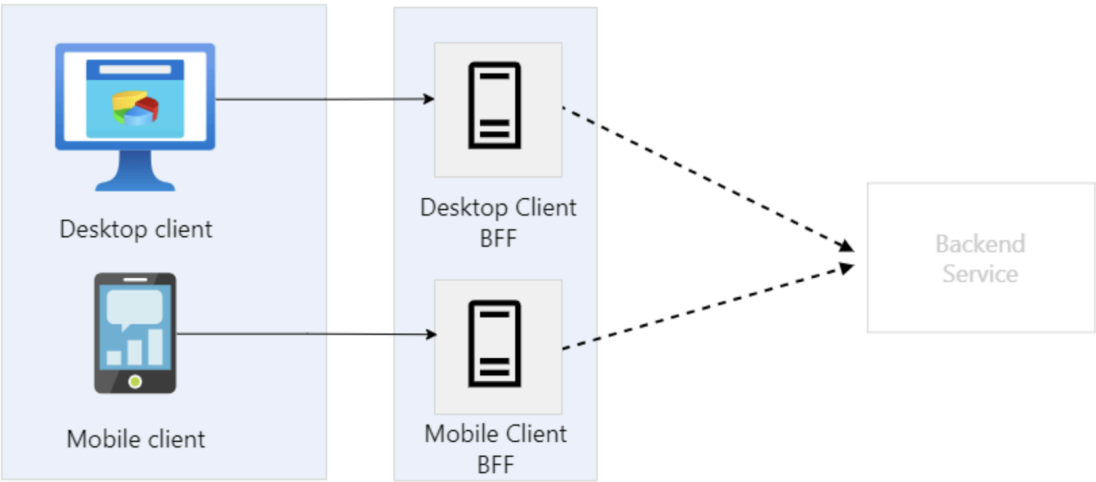


(optional) Reading: [What to consider when modernizing APIs with GraphQL on AWS | AWS Architecture Blog]

3. Backend for Frontend (BFF)

Instead of one gateway for all clients, each client type (web, mobile, IoT device like smart TV) has its own specialized “Backend for Frontend.” This microservice/gateway is optimized for that client’s specific needs.

Example: Netflix uses a mobile BFF that compress responses and handles offline logic for their device UIs (smart TVs, phones, …)



(optional) Reading: [Backends for Frontends pattern - Azure Architecture Center | Microsoft Learn]

Security Considerations

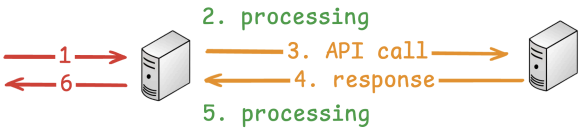
When exposing microservices or a gateway to the outside world:

- Use **HTTPS/TLS** to secure communication.
- Implement **Authentication & Authorization** at the gateway (often via **OAuth2, JWT** tokens).
- Possibly use **WAFs** (Web Application Firewalls) for advanced threat protection.
- **Rate limiting** and throttling to prevent **DDoS**.

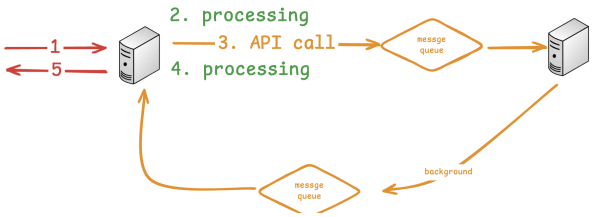
Microservice ⇔ Microservice Communication

Microservices communication can be

- **Synchronous:** Caller waits for immediate response (REST, gRPC, SOAP).



- **Asynchronous:** Caller does not wait — messages/events are queued and processed independently (Kafka, RabbitMQ).



Choosing the right model depends on business requirements, latency sensitivity, and reliability needs.

- **Tight Coupling vs. Loose Coupling**: The more a service depends on immediate responses from another, the more they're coupled in runtime behavior. Async communication can reduce runtime coupling but adds complexity and eventual consistency.

- **Scalability & Resilience**: Communication patterns have direct impact on performance and fault tolerance. Synchronous calls can cause cascading failures if a downstream service is slow or down (hence timeouts and circuit breakers). Asynchronous calls can decouple request and response in time, but introduce queues, complex flow, and potential data duplication.

1. Synchronous

- **RESTful APIs**
  REpresentational State Transfer (REST) is most popular due to simplicity and compatibility.

  Pros — Easy to build, test & debug (postman, curl). Widely known. Works over HTTP & JSON
  Cons — JSON is text based, so can be heavy in size.
  Serializing/Deserializing JSON is time consuming. Also, higher latency, because they're using HTTP
  JSON string: `{ "timestamp": "2025-03-28 08:54:21 +5:30 Asia/Kolkata" } ==>  100 bytes`
  Binary UTC timestamp: `8 bytes`

  ```
  fetch("service3.google.com/user?...", (response) => { ... }, (error) => { … })
  ```

- ~~**SOAP APIs**~~
  Older protocol using XML. Some enterprises still use it, especially in legacy or enterprise integrations, but it's less common in modern microservices.
  Not used for new services, because XML sucks *(unintuitive, verbose, full of edge cases, very time consuming to serialize/deserialize)*

  prediction: XML APIs will make a comeback because of LLMs (LLMs understand XML much better than they understand JSON)

- **Remote Procedure Calls** (RPC / gRPC)
  Either in your code, you can make a call, serialize data, send request, wait for response, get response, deserialize data, make a call.. Or, you can just make a function call.
  RPC abstracts out all the n/w things from your codebase – so that any API calls just feel like built in function calls in your code.

  gRPC was developed by Google.
  Provides high-performance, low latency communication by using HTTP/2 with binary serialization (Protobuf). Offers streaming support (client-stream, server-stream, bidirectional). Highly efficient and language-agnostic.
  - Ideal for internal service communication where performance and strongly typed contracts matter.
  - Adopted by large-scale systems (Netflix, Google)

  example: **gRPC** is a RPC developed by Google. It uses **Protobuf** for serialization.

(optional) Reading:
gRPC Crash Course - Modes, Examples, Pros & Cons and more

Note: gRPC often goes along with the sidecar pattern

- **Inter-Process Communication** (IPC) — extremely rare
  This requires the microservices to be deployed within the same server (as separate processes or containers) — hence, extremely rare.
  Communication primitives are provided by the Operating System and happen by sharing memory across processes, or by synchronous message passing b/w processes.

  (optional) Reading: Inter Process Communication (IPC) - Scaler Topics

  Stock exchanges have multiple microservices, and required extremely low overhead communication b/w the microservices.
  They also don't follow AC / CP, and instead follow AC (give up on P)
  Therefore, sometimes, stock exchanges deploy multiple microservices within the same server/container, and these services can communicate via IPC.

Considerations

- **Latency:** Synchronous calls add round-trip time. If a user request fans out to many microservices, the total latency can grow if not designed carefully.
- **Scaling:** Each synchronous call ties up resources. High concurrency can cause threads or connections to spike. We can mitigate this with non-blocking I/O or with concurrency limits. (Note: async-io is not the same as async communication)
- **Timeouts and Retries:** Must set sensible timeouts to avoid indefinite waiting. Retries can be used to handle transient failures.
  Can cause **retry storms** under heavy load. Must be combined with resilience patterns like ***Circuit Breaker***.

2. Asynchronous - Messaging & Event-Driven

Asynchronous communication decouples services, allowing independent scaling and improving resilience.

All asynchronous communication needs something (Message Broker) to *facilitate* the message passing

- **Message Queues** *(RabbitMQ, ActiveMQ, Azure Service Bus)*
  Services publish messages to a queue, consumers process them asynchronously. This is good for tasks that don't need immediate response

(e.g., background processing). A task is usually consumed by exactly 1 service.

- **Event-Driven / Pub-Sub** *(Kafka, SNS/SQS, Google Pub/Sub)*
  A service publishes an event when something happens (e.g., "OrderCreated"). Other services subscribe and react in real time. This decouples the producer from the consumers—producer doesn't need to know who is listening.
  One event can be consumed by multiple subscribers.

- **Event Streaming** *(Kafka Streams, Apache Flink)*
  For continuous data flows where events are processed in near real-time. Common in analytics or monitoring pipelines.
  Durable logs supporting event replay and analytics.

Pros

- Loose coupling (the producer doesn't get blocked waiting for consumer).
- Natural way to handle **"eventual consistency"** in microservices

Cons

- If you need immediate consistency for communication, you canNOT use message queues, as MQs are inherently asynchronous, which means, they will ONLY provide eventual consistency.
- Harder to track flows across services; need **distributed tracing**.
- Potential duplication of messages or out-of-order arrivals. Must handle **idempotence**.
  - for this, you have to configure at-least once, or at-most once, or exactly-once delivery.
  - You have to worry about the partitioning logic to ensure message order
- Typically requires a robust messaging or streaming infrastructure (clustered brokers), adding operational overhead.

In very large companies, the pros will outweigh the cons, which is why we will prefer MQs for microservice communication.

Does asynchronous mean I never need a response?

Not necessarily. You can design patterns where a response arrives on a separate channel (like a callback queue or a correlation ID mechanism). But it's more complex than direct synchronous replies.

Does asynchronous mean eventual consistency?

Yes, always!

If you need immediate consistency b/w two microservices, you must go back to synchronous communication!

Most real systems combine both synchronous & asynchronous patterns

- Query or read operations might be synchronous, so clients get immediate responses.
- State changes or background tasks can trigger asynchronous events. For example, placing an order might do both: respond immediately with “order received,” then publish an “OrderPlaced” event for downstream services to handle shipping, notifications, etc.

Common Pitfalls (both sync & async)

- **Overly Chatty APIs:** *Microservices which are communicating very frequently.* Solution: Combine requests or cache responses.
- **Blocking Calls:** Solution: Always implement timeouts, otherwise, it will cause Cascading failures
- **Complexity Overhead:** Choose communication models wisely, not everything requires asynchronous messaging.

Best Practices

- Design for failure (timeouts, retries, circuit breakers).
- Automate discovery (Kubernetes, Eureka).
- Keep APIs coarse-grained.
    - if you reveal a lot of internal details (deep-grained), then your clients will start “depending” on those details – later you won’t be able to change those details without breaking your client’s code
- Combine synchronous and asynchronous appropriately.
- Plan API versioning carefully.

Observability

Code does not always work fine. There are many things that may go wrong.

- There might be bugs in the code.
- There might be corner cases that are missed.
- There could be some problem with the machine your code is running on.
- There could be some problem with the network.

For small scale systems, basic monitoring (checking CPU, memory, and uptime) is sufficient.

However, as products grow and your architecture transitions to microservices, complexity skyrockets – simple monitoring becomes insufficient. Requests span multiple services,

latency problems become mysterious, and traditional dashboards tell you something is wrong, but not why.

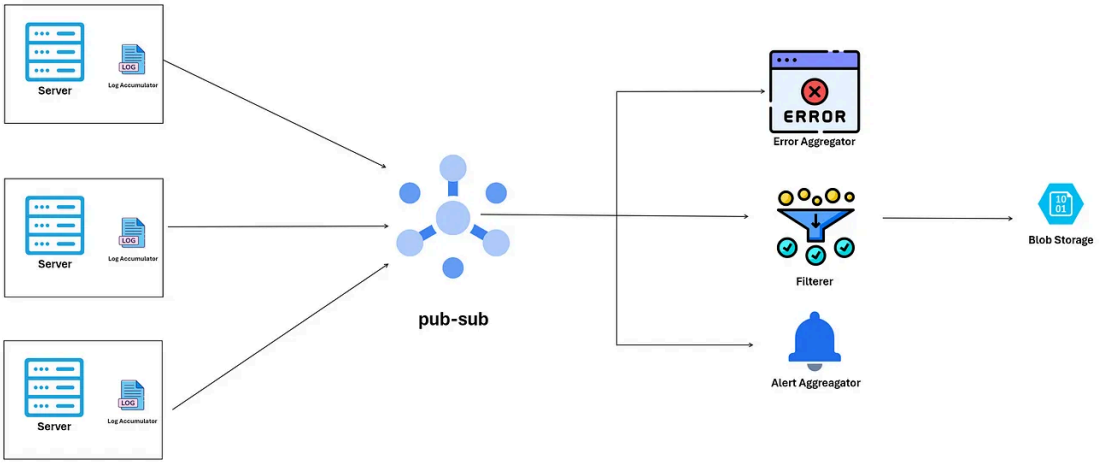
When something breaks in your system, you must

1. **Detect** that something is wrong — set alerts for anomalies in your metrics
2. **Diagnose** the issue & fix it — go through logs & traces
3. **Verify** that the fix actually works — the metrics should go back to normal

Observability aims to discover & help fix even "unknown" problems.

To achieve comprehensive observability, we rely on three critical components

- **Metrics**: Aggregated numeric data like latency, error rates, throughput.
- **Logs**: Detailed records of events in a structured format. They offer granular insight into application behavior.
- **Traces**: Visualizations of request paths across multiple microservices, pinpointing bottlenecks in distributed systems.



Metrics and Monitoring

Each microservice to expose a standardized `/metrics` endpoint. Metrics are tagged with labels (service name, instance ID, region) for detailed analysis.

Four Golden Signals

Introduced by [Google's Site Reliability Engineering team]

- **Latency:** How long do requests have to wait
    - this is typically handles by the middleware
- **Traffic:** A measure of how much demand is being placed on your system

- **Errors:** The rate of requests that fail, either explicitly (HTTP 5xx), implicitly (HTTP 2xx success response, but coupled with the wrong content), or by policy (if you committed to one-second response times, any request over one second is an error).

- **Saturation:** How "full" your service is. A measure of your system fraction, emphasizing the resources that are most constrained (e.g., in a memory-constrained system, show memory; in an I/O-constrained system, show I/O).

Monitoring these metrics provides deep insight into microservice health and helps swiftly detect anomalies.

What else to monitor?

- **User Behavior/Actions:** clicks, pages accessed, activities performed, time spent on page, …
- **User Experience:** ratings, NPS, sentiment, complaints, …
- **Revenue:** sales numbers, refund rates, CAC, …

Alerts

Automated notifications must be triggered when certain conditions in your system indicate a problem or potential issue.

- **delivery mechanism:** usually sent through via email, Slack, SMS, PagerDuty
- **Tiggering:** Triggered when specific thresholds or anomalies occur.
- Designed to prompt immediate attention or investigation.
- Good alerts are
    - **Actionable**: They clearly indicate what action is needed.
    - **Precise**: They avoid ambiguity and clearly state the affected service or metric.
    - **Relevant**: They only fire when the system genuinely needs intervention.
- Bad alerts are
    - **Noisy**: Too frequent, irrelevant, leading to *alert fatigue*.
    - **Ambiguous**: Lacking clear context or recommended actions.
    - **Low impact**: Triggered on minor issues that don't significantly affect the system or users.
- Best Practices
    - **Set appropriate thresholds**: Tune alerts to balance sensitivity and specificity.
    - **Escalation policies**: Define who gets alerted and escalation if the first responder doesn't act. *(prevent diffusion of responsibility)*
    - **Runbooks**: Document clear, actionable steps for handling alerts.
    - **Regular review**: Regularly audit alerts to avoid false positives and negatives.
- Example alerts
    - alert if `5xx error rate > 1% for 5 minutes`
    - alert if `average hdd utilization > 80%`
    - alert if `nps < 30`

Popular Tools

- **Prometheus** — A pull-based monitoring system widely adopted in the microservices ecosystem.
- **Grafana** — Visualizes Prometheus data, turning raw metrics into actionable insights.
- **PagerDuty**
- **Opsgenie**

Which is better—push or pull for metrics?

Prometheus typically does pull ("scraping"). Some systems do push (StatsD).
Both work — what matters is consistent labeling and high reliability.

How often should I scrape metrics?

Common intervals are 15s, 30s, or 1m. More frequent scraping can give finer-grained data but can be more resource-intensive.

Can I store metrics long term for historical analysis?

Yes, but you might need a long-term storage solution (e.g., remote storage in Thanos, InfluxDB, or logs-based solutions). Prometheus itself is optimized for recent data (defaults around a few weeks).

You must have some archiving policy in place.

Logging

Logs are essential for diagnosing issues

- **Capture:** logs must be captured from all independent services & individual servers
- **Structure:** logs must be standardized and structured
    - JSON formatted logs with timestamps, correlation IDs, and key-value pairs

```
{
  "correlationId": "ABCDEF123456",
  "timestamp": "2025-03-28T09:20:45.000Z",
  "level": "INFO",
  "service": "payment-service",
  "message": "Payment processed | successfully",
  "paymentId": "PAY12345",
  "userId": "USER67890",
}
```

Don't do the following
~~ABCDEF123456 | 2025-03-28T09:20:45.000Z | INFO | payment-service | Payment processed | successfully~~

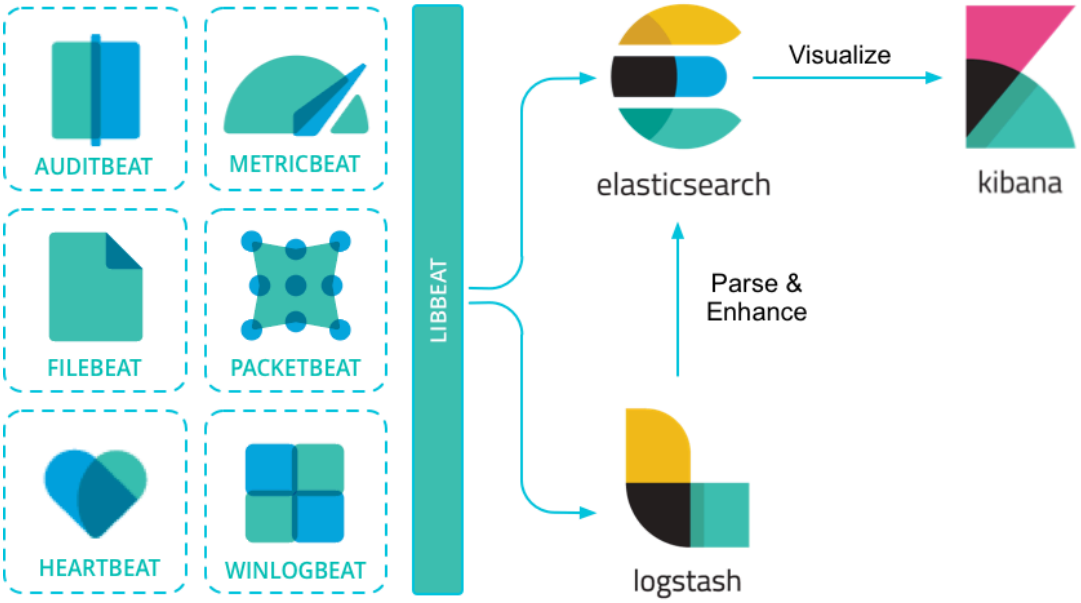
```
Use JSON, because it is both human readable, and machine
parsable.
```

- **Aggregation**: individual logs scattered across multiple servers are inefficient. You can't ssh into 1,000 different servers. Centralization gathers all logs into a unified, ***searchable*** location.

Popular Tools

1. **Log Shippers** — collect logs from each instance/container and send them to a central location. eg, Filebeat, Fluent Bit, Logstash
2. **Storage & Search**: store logs and provide a search interface. eg, Elasticsearch, Splunk, Datadog
3. **Visualization**: lets you query logs, build dashboards, and set alerts. eg, Kibana (in the ELK stack) or Splunk's UI

- **ELK stack** — Elasticsearch, Logstash, Kibana



- Cloud solutions — AWS CloudWatch, Splunk, Datadog

How long should the logs be stored for?

Don't delete them ever! Just archive them periodically (move them out of the search database, analytics, ..) — because storage is cheap

How do we handle personally identifiable information (PII) in logs?

Mask or avoid logging PII. Use compliance-friendly approaches (e.g., hashing user IDs).

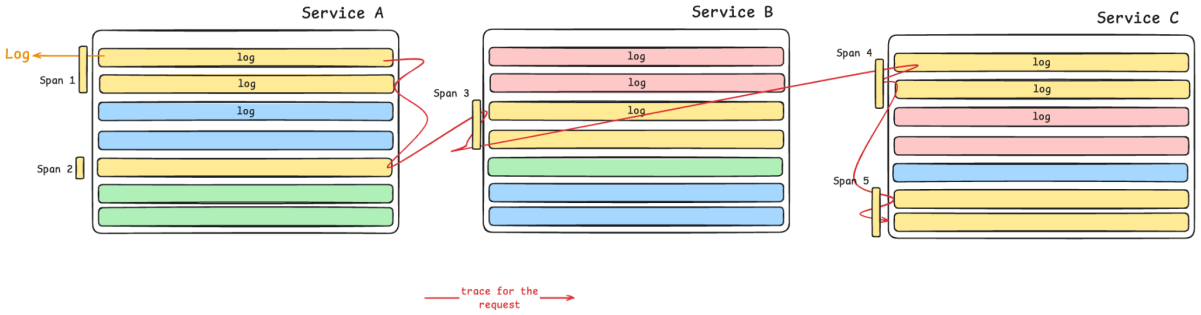
Don't log sensitive data like passwords & tokens, API keys, env variables, ..

Distributed Tracing

Metrics highlight that a problem exists. Logs record discrete events.

But distributed tracing uniquely provides detailed insight into request flows across multiple services, highlighting latency bottlenecks and pinpointing failures.

1. **Traces**
   - A trace represents the entire journey of a single user request as it travels through multiple services.
   - **Trace ID / Correlation ID / Request ID**
     - A unique ID generated for a request as it first enters the API gateway (request entry point) — this trace id will be stored in the request headers
     - **Context Propagation**
       1. Each service must mandatorily include the trace-id for each log line it produces
       2. Each service must mandatorily pass-on the trace-id for any subsequent API calls made for this request
       3. happens via HTTP headers `x-b3-traceid` or W3C `traceparent`
     - Trace IDs are used to link logs from multiple services involved in the same request.
     - A trace typically resembles a tree or waterfall view, visualizing how the request flows from service to service.
2. **Spans**
   - A span represents a specific operation or unit of work within the trace.
   - Each span has a unique **span ID** and references its parent span (the operation that triggered it).
   - Spans include information such as
     - Operation name (e.g., database call, HTTP request).
     - Start and end timestamps (to measure latency).
     - Service or component that executed the operation.
     - Relevant metadata or logs for deeper context.



Scaling Observability

Observability can quickly produce vast amounts of data. Techniques like sampling, log retention policies, and tiered storage (hot/cold) are used to manage scale and cost.

Should I build my own observability stack or use a managed service?

Depends on in-house expertise and scale.

Many companies start with open-source (Prometheus, Jaeger) and might move to managed offerings (Datadog, New Relic) if they prefer turnkey solutions.

Most of these managed service providers have very generous free tiers!

Resources & Case Studies (optional)

- Achieving observability in async workflows | by Netflix Technology Blog
- Building Netflix's Distributed Tracing Infrastructure | by Netflix Technology Blog
- Edgar: Solving Mysteries Faster with Observability | by Netflix Technology Blog

Resilience

With microservices, we've gained autonomy and scalability. But we've traded the relative safety of the monolith for the inherent unreliability of the network and distributed state.

Services *will* fail. Networks *will* glitch. Deployments *will* introduce bugs. That's a fact!

Therefore, we need to make our systems resilient.

Resilience is not about *preventing* failure (that's impossible), but about *containing* failure and ensuring the system *as a whole* remains available and degrades gracefully, rather than collapsing completely.

Think of it like designing a ship. You can't guarantee you'll never hit an iceberg, but you *can* design watertight compartments (*bulkheads*) so one breach doesn't sink the whole vessel.

Common Failure Types

To design resilient systems, we first need to understand common failure scenarios

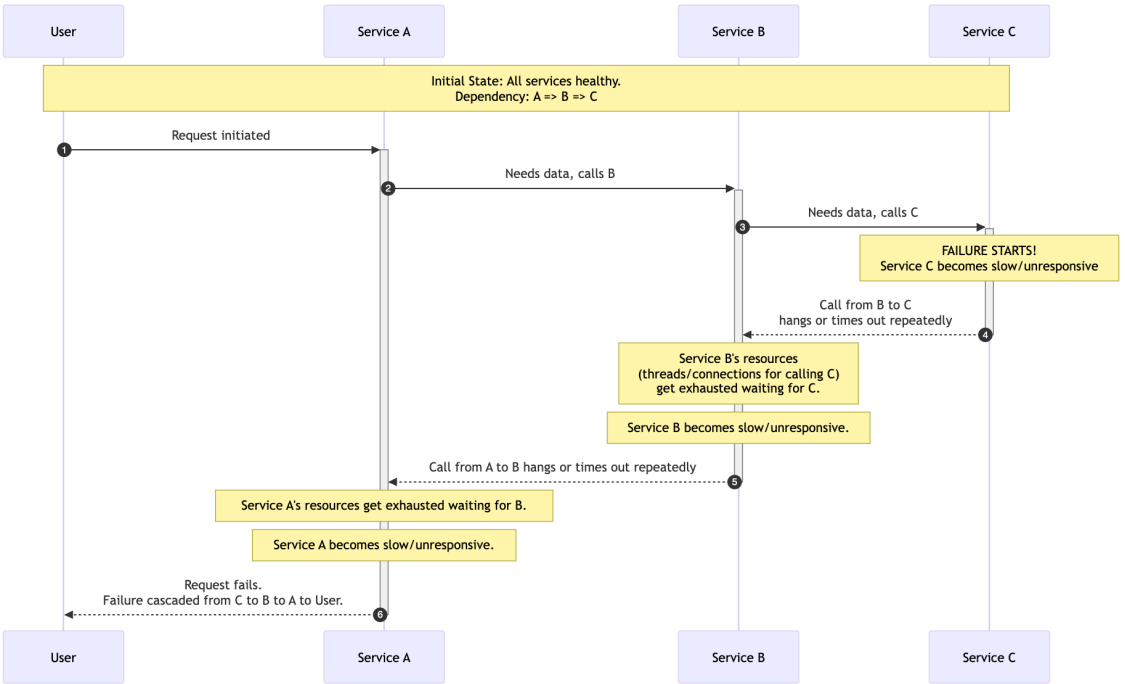
- **Service Unavailability:** Instance crash (out of memory, bugs, cosmic events), deployment issue, underlying infrastructure failure.
- **Network Issues:** Timeouts, packet loss, high latency, DNS problems.
  - A congested network can behave like a downed network
  - The network doesn't have to fail completely for it to cause problems
- **Service Overload/Slowness:** A dependency becomes slow due to load, inefficient queries, resource contention, etc.

These individual failures can lead to dangerous systemic effects..

Cascading Failures

When failure of one service causes the failure of other services *(the ones that depend on the failed service)*

In the below example, all calls are synchronous



1. if Service C was not down, but was just facing some internal error, then it would respond to requests with an error (like 5xx, or 4xx, or 3xx). In this case, the requests will not time out — service B would receive responses, it's just that those responses will have an error code.
    a. **That's not the case.** Service C is down — it simply doesn't respond – or responds so late that the requests times out.
2. Service B has to maintain some resources in order to handle the requests
    a. each time B gets a request
        i. spins up a new thread
        ii. sends a request to C
            1. takes RAM to store the connection data
            2. takes a port
        iii. it has to wait for a response
    b. B is getting multiple requests in parallel
    c. Because lots of requests to C are timing out, all the requests that have to come to B have to wait until the timeout happens
    d. if any new requests come during this time, B doesn't have enough threads/ports/RAM to handle those requests – because it is still waiting for the earlier requests to come back from C

This causes a Domino Effect!

To resolve this, we must detect that service C is down. If service B knows that service C is down, it can ***reject requests without incurring failure***!

Basically, replace the timeout with an error code.

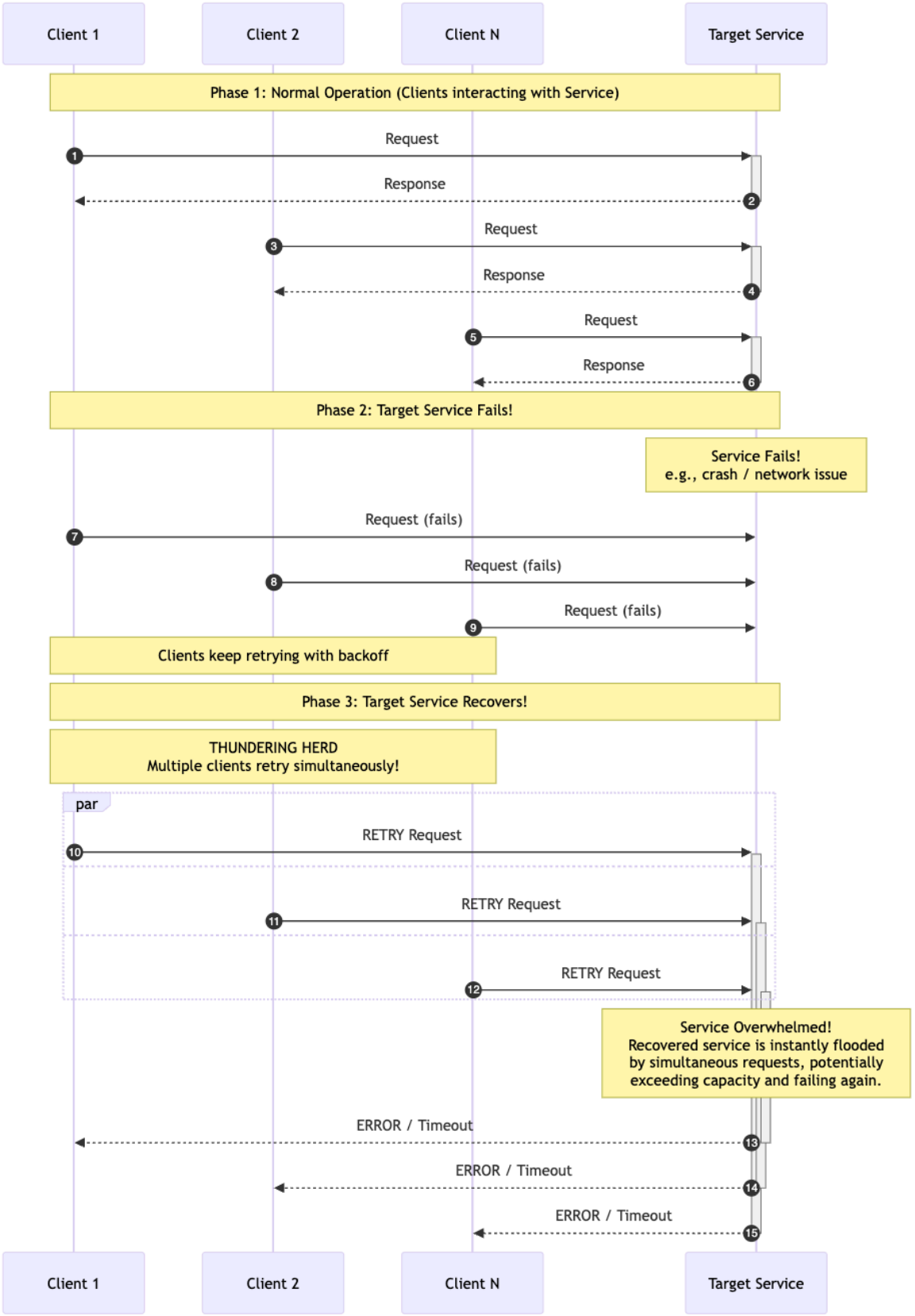
Does Cascading Failure happen only in Synchronous communication?

Mostly, yes. But it can also happen (rarely) with asynchronous communication! This typically happens when the Message Queue becomes overloaded

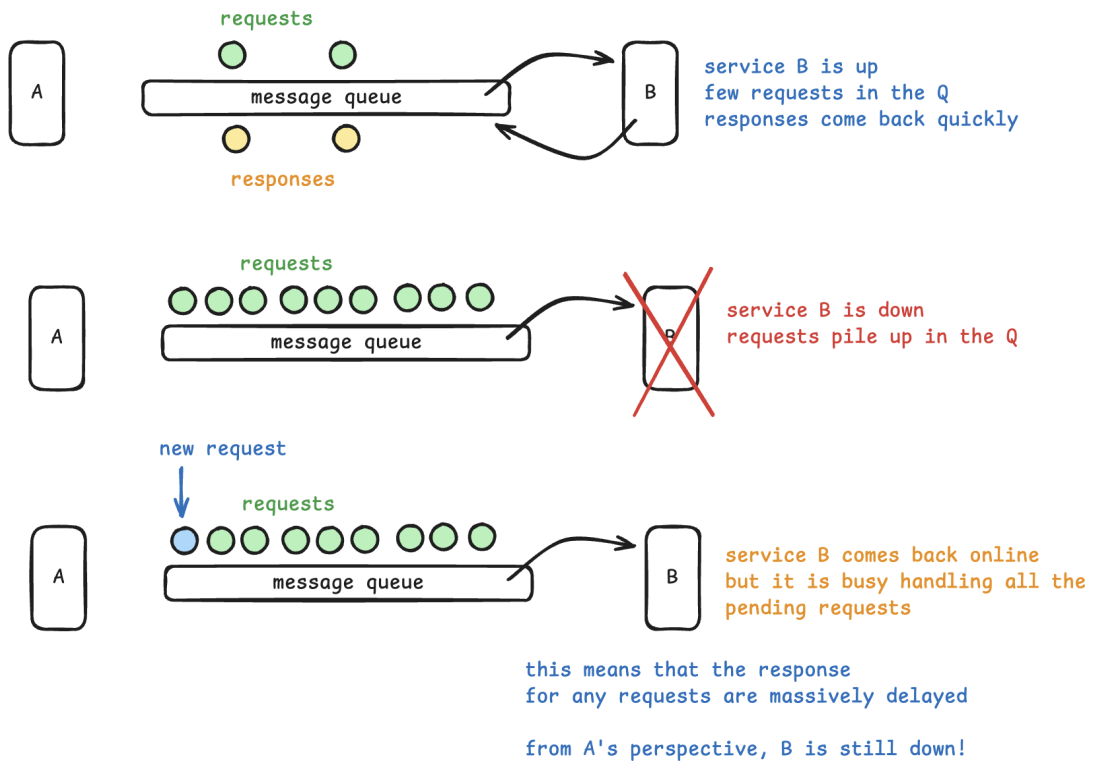
Thundering Herd

1. Imagine a service fails and while many other services are trying to call it.
2. When the service finally recovers
    - *Instantly*, all waiting clients retry simultaneously
    - this overwhelms the just-recovered service
    - and causes it to fail again immediately!

This often happens after timeouts expire across many clients, or when a load balancer brings a recovered instance back into rotation.



Note that Thundering Herd can happen even in async communication!



Understanding these failure modes is crucial for appreciating *why* the following patterns are necessary.

Basic Defenses

Timeouts

- *Every* network call MUST have a timeout. Non-negotiable.
- Protects the caller from indefinitely waiting for a slow/dead dependency.
- Tune carefully based on expected latency (P95/P99) and business impact.
  - "How NOT to Measure Latency" by Gil Tene

Smart Retries

- **Handle *transient* failures:** temporary network glitches, brief load spikes — doesn't mean that the world has ended – just retry
- **Avoid Naive Retries:** Immediate, repeated retries amplify load and cause Thundering Herds.
- **Use Exponential Backoff:** Increase delay between retries (100ms, 200ms, 400ms...).
- **Add Jitter:** Randomize the backoff delay slightly to prevent synchronized retries.
- **Limit Retries:** Usually 2-3 max.
- **Mandatory Idempotency:** Ensure retried operations don't have unintended side effects.

Timeouts & Retries only help with transient issues. If a service is persistently slow or down, retries will eventually fail and still consume resources.

We need advanced defense mechanisms …

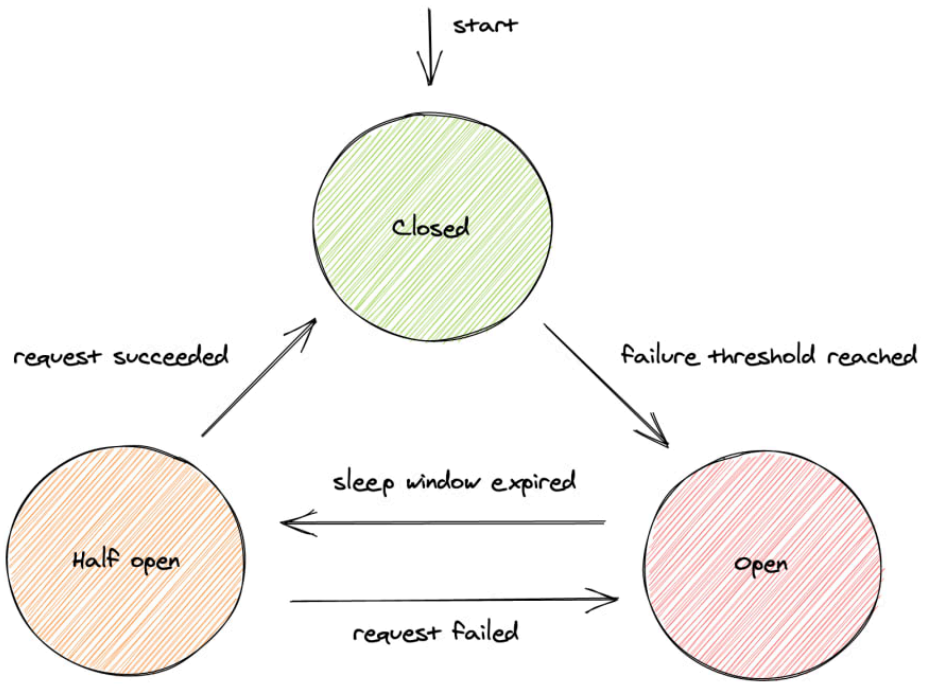
Circuit Breaker

This is arguably the most critical pattern for preventing **Cascading Failures** & **Thundering Herd.**

Idea: Add a circuit breaker, which acts as a protective proxy around calls to a dependency — It monitors failures and trips "open" if the dependency seems unhealthy, preventing further calls for a while.

State Machine

For every service (not server) inside your system, you will maintain a “state”
There are 3 possible states — closed, open, half-open — and the services’ state can transition b/w these 3.



1. **Closed**
   - Initial state.
   - circuit closed == current is flowing (everything is fine)
   - Allows requests through. Monitors success/failure/latency.
   - *Transition:* If failures exceed a configured threshold, say `>50% errors/min`, or `average latency > 1s`), it trips to **Open**.

2. **Open**
   - Fail Fast — requests fail immediately without attempting the network call.
   - circuit open == 0 current flow
   - Often triggers a fallback.
   - *Transition:* After a configured reset timeout, say `60s`, transition to **Half-Open**

3. **Half Open**
   - Allows a very small number of "probe" requests through to the dependency.
   - *Transition:*
     - If probe requests succeed
       - Assume dependency has recovered.
       - Transition: back to **Closed**.
     - If probe requests fail
       - Assume dependency is still unhealthy.
       - Transition: back to **Open** (restarting the reset-timeout).

```
Service A:
    def handle_request(req):
        ...
        data = circuit_breaker.make_call("B", ...)
```

```
        ...

Circuit Breaker:
    [maintains the “state” for each service]

    def make_call(service, data):
        state = service_states.get(service)
        if state == ‘closed’:
            // actually make n/w call to service B
        if state == ‘open’:
            return 5xx
            (if time-since-state-open > sleep_timeout, then mark
state as half-open)
        if state == ‘half-open’:
            p = random()
            if p < 1%:
                // actually make n/w call to service B
            else:
                return 5xx

Service B:
    def handle_request(req):
        ...
```

Infrastructure Requirements

You will need to store the “states” for each service in some central location — Redis server.

This doesn’t always need to be central (but it is a lot of times) — each service can maintain its own independent view of the states of its dependencies (circuit breaker can be implemented as a library / sidecar).

Implementation Considerations:

- **Scope:** Typically applied per-dependency, sometimes per-operation within a dependency.
    - Note: simple healthcheck is NOT sufficient
    - healthcheck / heartbeat only checks if the service is “up” as a whole.
    - what if 1 endpoint is misbehaving, but other endpoints of that service are fine —- some client could depend on that 1 endpoint
- **Thresholds:** Need careful tuning based on baseline metrics (error rates, latency percentiles).

    - Static thresholds can be brittle.
    - Adaptive thresholds are better but more complex.
- **Monitoring:** Critical to monitor the state of your circuit breakers! A breaker stuck at Open indicates a persistent downstream problem that needs investigation. Put an alert on state transitions.

Popular Implementations: Resilience4j, Polly, Istio/Linkerd

Advantages of Circuit Breaker

1. **Prevents Cascading Failures:** Stops upstream services from getting bogged down calling a known-bad dependency.
2. **Reduces Latency during downtime:** Fail Fast — Resources (threads, connections) are released quickly. Latency is reduced for users/callers when a dependency is known to be down.
3. **Prevents Thundering Herd:** Allows recovery by giving the struggling dependency time to recover without being hammered by continuous requests

Cons of Circuit Breaker

1. Requires additional infrastructure to maintain service states
2. Needs additional code
3. Adds a small amount of network traffic

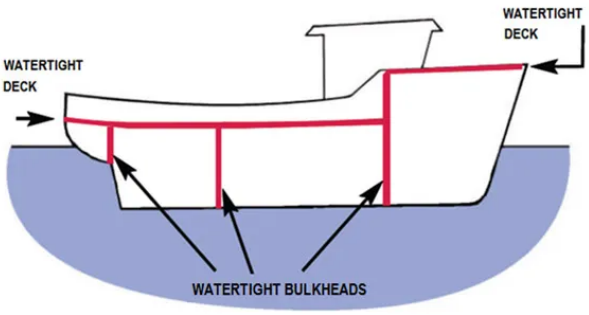
Circuit breaker is like automatic “on-vacation” email replies. You don’t want someone emailing you to have to wait and think you’re an asshole for not replying.

Failing fast is good!

Bulkhead (Resource isolation)

While Circuit Breakers stop calls *to* a failing dependency, Bulkheads prevent a failing dependency from consuming *all* resources *within* the calling service.

Idea: Isolate resources used for calls to different dependencies, preventing failure in one from affecting others.



Isolation

- **Thread Pools:** Assign a dedicated, fixed-size thread pool for calls to Service A, another for Service B, etc.
    - If Service A becomes extremely slow, it might saturate *its* thread pool.
    - However, calls to Service B, using a separate pool, continue unaffected.
- **Connection Pools:** Similar isolation for database connections or other pooled network resources.
- **Memory:** Less common, but isolating memory buffers or queues per dependency.
- **Concurrency Limits** (Semaphores)**:** Limit the number of *concurrent* requests allowed to a specific dependency, regardless of threads.

Benefits

- **Limits Blast Radius:** Failure or slowness in one dependency doesn't starve resources needed by others. Improves overall system stability.
- **Prevents Resource Exhaustion:** Protects the calling service itself from running out of threads, connections, or memory due to one bad actor.

Implementation Considerations

- **Resource Sizing:** Requires careful sizing of pools/limits based on expected load and latency. Oversizing wastes resources; undersizing creates bottlenecks.
- **Overhead:** Managing multiple thread pools can add some context-switching overhead compared to a single large pool.
- **Where to Apply:** Most beneficial when calling external dependencies with variable or potentially high latency. Often used in conjunction with Circuit Breakers.

Graceful Degradation & Fallbacks

Okay, our Circuit Breaker tripped, or a call timed out after retries. The call *failed*.

What now? Just returning a `500: Internal Server Error` is often not the best user experience.

Note: if A → B, but B is down, the A should get a 5xx error immediately. But the user should not get a 5xx error because that is bad experience.

Graceful Degradation: Design your service to provide *partial* or *alternative* functionality when dependencies are unavailable.

Fallback Strategies

- **Return Cached Data:** Serve the last known good response from a cache.
- **Return Default Value:** Show generic recommendations, zero balance, default settings.
- **Return Partial Response:** If calling Service A and B, and B fails, return data only from A if that's still useful.
- **Queue for Later Processing:** For background tasks, queue the request to be retried when the dependency recovers — but only until a certain limit, otherwise, it will cause thundering herd
- **Return Degraded Data:** Provide a lower-fidelity version (e.g., lower-resolution image).
- **Informative Error Message:** If no fallback is feasible, provide a clear message like "Unable to load your preferences right now, please try again later."

Key: Identify critical vs non-critical dependencies. Failure of a critical dependency might necessitate an error, while failure of a non-critical one should ideally trigger a fallback allowing the core functionality to proceed.

Chaos Engineering Principles (advanced defense)

How do you *know* your timeouts, retries, circuit breakers, bulkheads, and fallbacks *actually work*? You test them under realistic failure conditions.

Idea: Build confidence by *intentionally* injecting controlled failures into your system (ideally production, carefully!) to verify resilience mechanisms.

[Home - Chaos Monkey]

- Terminate instances, inject latency/errors, stress CPU/memory.
- Methodology
  1. Define steady state (normal behavior).
  2. Formulate hypothesis — like “Injecting `500ms` latency into Service B will cause Service A's circuit breaker to trip, and Service A will serve cached data”.
  3. Design experiment (scope, blast radius, duration).
  4. Run experiment & observe (monitor metrics, logs, breaker states).
  5. Analyze results & improve system if hypothesis fails.
- Starting Safely
  1. Begin in dev/staging, small blast radius, non-critical services, game days before automating.

Chaos Engineering turns "we think it's resilient" into "we know it's resilient because we tested it."

Resources (optional)

- **Book:** [*"Release It! Design and Deploy Production-Ready Software"*] — *The* classic text on stability patterns.
- **Libraries:**
  - Resilience4j (java) Introduction
  - Polly (.net) GitHub - App-vNext/Polly
- **Service Mesh Docs:**

    - Istio / Circuit Breaking
    - Retries and Timeouts | Linkerd
- **Chaos Engineering:**
    - Principles of chaos engineering
    - A curated list of awesome Chaos Engineering resources
    - Proactively improve reliability | Gremlin Blog

Doubts

Sidecar Pattern

How Atlassian reduced latency by 70% by using Sidecar pattern and when should you …

Imagine that you're responsible for Service A, and you've lot of other services as clients.
To ensure proper requests, you've defined some "best practices"

- how to call the service
- what should be in the payload
- max payload size
- how frequently to retry
- rate limits

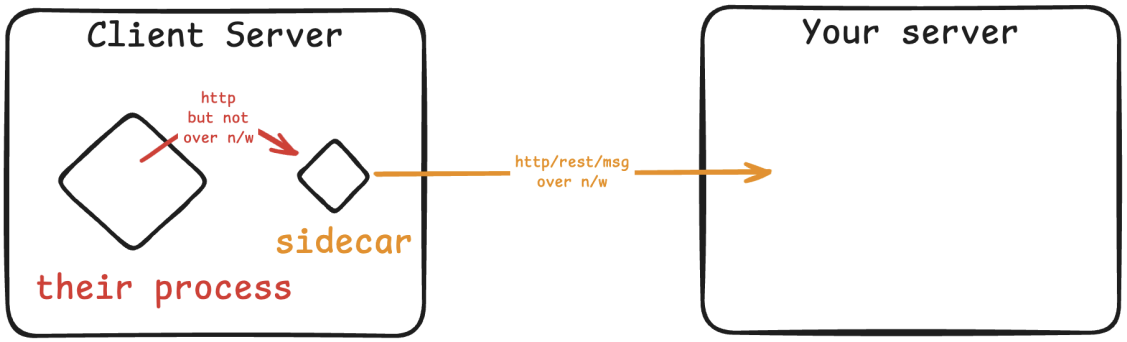
You can just document these practices & inform the other teams about them.
Does this guarantee that other team will follow them?

Absolutely no — they will whatever they want *(mah service, mah rules!)*

If you want to enforce these best practices, you can create a "sidecar" as a

- (not recommended) library that the other person can install in their codebase — not recommended because you will have to publish this library in multiple languages (because your clients can use any tech stack)
- (recommended) separate executable that runs in a process in the same server as your client's service

This sidecar will now act as a proxy — you client will make a request to the sidecar, and the sidecar will "enforce" the best practices while forwarding the call to your service.



Apart from this, other cross-cutting concerns that span multiple services (logging, metrics, …) can also be implemented as sidecars.

For example, to report the CPU, HDD, RAM utilization metrics, you can implement a separate “resource monitoring” process and install it on all servers.

NextJs — built by Vercel —- cloud hosting.

Nextjs has an pre-fetching feature.

Nextjs security hole — This Is Bad!

Q: How do we handle common utils across microservices?

1. No, your utils are not THAT common!
    a. Allow each microservice to have their own custom version of it.
    b. DRY principle matters only within the codebase, not across it
    c. The issue with duplicity is that if you want to modify it, you will have to update it everywhere, otherwise, things will be inconsistent.
        i. If your utils contains critical business logic, then they’re not really utils.
        ii. If they don’t contain critical business logic, then it’s okay to duplicate them
2. You can use 3rd party libraries that will provide you these utils
3. **You can have your own “internal” libraries that provide you these utils.**
    a. The team that manages this library will be separate
    b. Other teams will simply import this library
    c. In large companies this is extremely common!

```
class UserController:
    @get('/users)
    def get_user(request):
        start_time = time.time()
        id = request.user_id
        user = fetch_user_from_db(id)
        …
        end_time = time.time()
        latency_service.log_request_time(
            'UserController.get_user',
             end_time - start_time
        )
        return user

   @get('/users/profile-pic)
    def get_user_dp(request):
        start_time = time.time()
        id = request.user_id
        user = fetch_user_from_db(id)
        …
        end_time = time.time()
        latency_service.log_request_time(
            'UserController.get_user',
             end_time - start_time
        )
        return picture
```

Instead of setting this up at every handler, you can just set this up in the middleware